Nirma University
Institute of Technology



Course Code :- 6CS102CC26
Course Name :- Data Structure

Innovative Assignment 2

MatchX – Order Book Simulator

Student Names

Kelly Gandhi
Sujal Patel

Student Roll No.

26MCA019
26MCA043

Faculty In Charge

Prof. Nirav Patel

Contents

1. Abstract

2. Introduction and problem statement

3. Objectives

4. Key terms

5. System design

6. Data structures used

7. Algorithms

8. Time and space complexity

9. User interface

10. Testing

11. Performance benchmark

12. Limitations and future scope

13. Conclusion

1. Abstract

A stock exchange keeps an **order book** for every share: a list of people who want to buy (bids) and a list of people who want to sell (asks). Its **matching engine** pairs them using **price–time priority**: the best price is served first, and among orders at the same price, the one that arrived first wins.

MatchX is a single-page web application that simulates this. It runs a live market of **60 stocks** with automated traders, and gives the user a demo account with ₹10,00,000 of virtual cash. The user can place limit and market orders, then update or cancel them. Each stock's order book is stored as **two binary heaps**, a max-heap of buys and a min-heap of sells. Supporting structures include a hash map for order lookup, lazy deletion for cancellations, a circular buffer for the audit trail, and dynamic arrays for trade history.

A built-in benchmark compares the heap engine with a sorted-array engine on the same order stream. With 1,50,000 orders, the heap engine was about **2.8× faster** in Chromium, and both engines produced exactly the same 37,872 trades.

2. Introduction and problem statement

When thousands of orders arrive every second, an exchange must answer two questions quickly and fairly: *what is the best buy and best sell price right now?* and *which waiting order should trade next?* Searching a plain list for the best price takes time proportional to the number of orders. Keeping a sorted array makes finding the best price fast, but every insertion has to shift elements.

Problem statement: Design and implement an order matching engine that keeps a separate order book for each stock. The engine must:

- always match the best-priced buy with the best-priced sell;
- break ties by arrival order (first in, first out);
- support partial fills, cancellations and modifications;
- check that the trader has enough cash or shares before accepting an order.

It should also present all of this in a realistic trading interface, and measure its performance against a simpler design.

3. Objectives

- Use a **binary heap** as a priority queue so the best price is available in O(1) time, and insertion and removal take O(log n).
- Enforce **price–time priority** with a sequence number used as a tie-breaker in the heap comparator.
- Support **limit orders**, **market orders** (immediate-or-cancel with 5% price protection), **partial fills**, **cancel** and **modify**.
- Make cancellation O(1) using a **hash map** and **lazy deletion**.
- Keep a bounded **audit trail** of the latest 2,000 events using a **circular buffer**.
- Validate funds: buy orders lock cash and sell orders lock shares, so the same money or shares cannot be used twice.
- Compare the heap engine with a **sorted-array** engine using a reproducible benchmark.

4. Key terms

Term	Meaning in MatchX
Bid / Ask	A buy order / a sell order.
Best bid / best ask	The highest buy price / the lowest sell price. These sit at the top of the two heaps.
Spread	Best ask − best bid, shown in the middle of the order book.

Limit order	Buy or sell at a stated price or better. If it can't trade now, it waits (*rests*) in the book.
Market order	Trade immediately at the best available price. MatchX caps it at 5% from the best price, and any quantity left over is cancelled (immediate-or-cancel).
Crossing	A buy crosses when its price is ≥ the best ask; a sell crosses when its price is ≤ the best bid. Only crossing orders trade.
Partial fill	Only part of an order's quantity traded. The rest stays in the book.
Price–time priority	Better price first; at equal price, earlier arrival first.

5. System design

5.1 Order life cycle

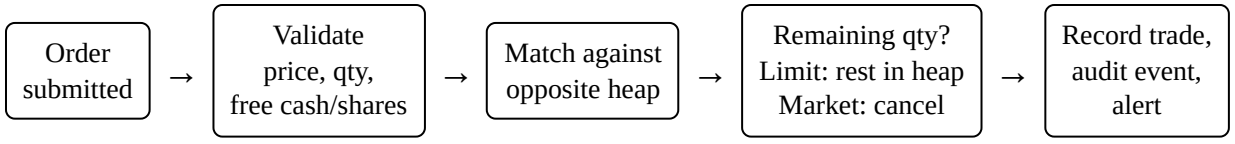


Figure 1 – How every order flows through the engine. Rejected orders stop at validation and are logged.

5.2 Modules

Module	Responsibility
`Heap` class	Generic binary heap with a comparator. Provides `push`, `pop` and `peek`.
`Ring` class	Fixed-size circular buffer for the audit trail.
`Exchange` class	The matching engine. It holds the order books, the order map, trades, cash and holdings. Methods: `submit`, `match`, `cancel`, `modify`, `levels`, `reserved`, `worth`.
`seed()` and `bot()`	Create 80 price points and 12 resting orders on each side for every stock, then generate random bot orders (1–80 per second, set by the speed slider).
`runHeap` / `runArr`	Two stripped-down engines used only by the benchmark.
React components	Landing page, watchlist, candlestick chart, order book, order ticket, and tabs (Holdings, Orders, Open orders, Trade history, Audit trail, Benchmark, Data structures).

5.3 Order record

```
{ id, sym, side: "BUY" | "SELL", price, qty, rem,   // rem = quantity still open
  seq,                                             // arrival number for time
priority
  status: "OPEN" | "PARTIAL" | "FILLED" | "CANCELLED" | "REJECTED",
  user: "YOU" | "BOT", t, rest, fv }               // fv = filled value, for
average price
```

6. Data structures used

Structure	How MatchX uses it	Cost
Binary heap	One max-heap of buys and one min-heap of sells per stock. The top is the best price.	Insert/remove O(log n), best price O(1)
Sequence number	Tie-breaker in the comparator, so equal prices are served first in, first out.	O(1) per comparison
Hash map (`Map`) and set (`Set`)	Order ID → order record, used for cancel, modify and status. A set holds the IDs of the user's open orders.	O(1) average
Lazy deletion	A cancelled order is only marked. It is removed when it reaches the top of its heap.	Cancel O(1)
Circular buffer	The audit trail keeps the latest 2,000 events in fixed memory.	Add O(1)
Dynamic array	Executed trades and per-stock price history for the charts.	Append O(1) amortised
Sorted array	Baseline engine used only in the benchmark (binary search plus `splice`).	Insert O(n)

6.1 Binary heap with price–time priority

The heap is a complete binary tree stored in an array: the children of index *i* are at $2i+1$ and $2i+2$, and its parent is at $(i-1)/2$. The same class serves both sides of the book. Only the comparator changes:

```
const bidC = (x, y) => y.price - x.price || x.seq - y.seq;  // higher price first,
then earlier
const askC = (x, y) => x.price - y.price || x.seq - y.seq;  // lower price first,
then earlier

push(x) {                        // sift up: O(log n)
  a.push(x); let i = a.length - 1;
  while (i > 0) { const p = (i - 1) >> 1;
    if (c(a[i], a[p]) < 0) { [a[i], a[p]] = [a[p], a[i]]; i = p; } else break; }
}
pop() {                          // move last to root, sift down: O(log n)
  const top = a[0], last = a.pop();
  if (a.length) { a[0] = last; /* swap with the better child until in place */ }
  return top;
}
```

Why the sequence number matters: two sell orders at ₹100 compare as equal on price, so `x.seq − y.seq` decides the order, and the one that arrived first is matched first. Testing confirmed this (section 10, test 3).

6.2 Lazy deletion

Removing an arbitrary element from the middle of a heap is awkward. When an order is cancelled, MatchX only sets its status to `CANCELLED` and removes it from the hash map, which takes O(1). The order stays inside the heap array. During matching, the engine checks the top: if that order is no longer live, it is popped and skipped.

```
let top = opp.peek();
while (top && !this.live(top)) { opp.pop(); top = opp.peek(); }   // drop dead
orders
```

6.3 Circular buffer (audit trail)

Every event (ACCEPTED, EXECUTED, MODIFIED, CANCELLED, REJECTED) is written to a ring of 2,000 slots. The write index wraps with `(i + 1) % size`, so the newest event overwrites the oldest and memory never grows. `newest(k)` reads backwards from the write position for the Audit trail tab.

7. Algorithms

7.1 Submitting an order

1. Create the order with a new `id` and `seq`.
2. Reject it if the price is ≤ 0 or the quantity isn't a positive whole number.
3. For the user's orders, compute the **reserved** amounts: cash locked by open buy orders and shares locked by open sell orders. Reject a buy if *cash − reserved cash < price × qty*. Reject a sell if *held − reserved shares < qty*.

4. Store the order in the hash map, log it as ACCEPTED, and call `match`.
5. If quantity remains: a market order cancels the rest (immediate-or-cancel). A limit order is pushed into its side's heap and waits.

7.2 Matching

```
while (order.rem > 0) {
    top = best live order on the opposite heap        // lazy-delete dead tops
    if (!top || price does not cross) break
    q = min(order.rem, top.rem);  tradePrice = top.price   // the resting order
sets the price
    order.rem -= q;  top.rem -= q;  lastPrice = tradePrice
    update the user's cash, holdings and cost if they are on either side
    record the trade; write EXECUTED to the audit trail; alert the user if their
resting order filled
    if (top.rem == 0) { pop top; mark FILLED }
}
if (order.rem == 0) mark FILLED
```

The trade always happens at the **resting (older) order's price**, as on a real exchange. A buyer who bids above the best ask therefore gets the better price.

7.3 Market orders

A market buy is sent as a limit order at *best ask × 1.05*, and a market sell at *best bid × 0.95*, with the immediate-or-cancel flag. It fills against everything available within 5% and cancels the rest, which protects the user from a very bad price in a thin book. The confirmation dialog shows this as the "worst price".

7.4 Cancel and modify

- **Cancel:** look up the order in the hash map, mark it CANCELLED and remove it from the map. This is O(1); the heap entry is dropped later.
- **Modify, same price and smaller quantity:** the order is changed in place and **keeps its time priority**.
- **Modify, new price or larger quantity:** funds are rechecked, the old order is cancelled and a new order is submitted with a new ID. It **loses its time priority**, which is fair to traders who were already waiting at that price.

7.5 Order book display

`levels()` walks the side's heap array, skips dead orders, and groups live orders by price into (price, total quantity, number of orders). It then sorts the levels and shows the best 9 on each side, with depth bars proportional to quantity.

8. Time and space complexity

n = number of orders resting on one side of one stock's book; *u* = number of the user's open orders; *k* = trades created by one incoming order.

Operation	Time	Reason
Best bid / best ask	O(1)	Top of the heap (`peek`).
Add a resting order	O(log n)	Heap sift-up.
Match an incoming order	O(k log n)	Each fully filled resting order is popped (sift-down).
Funds check (`reserved`)	O(u)	Loops over the user's open orders.
Cancel	O(1)	Hash map lookup and mark (lazy deletion).
Modify (smaller quantity)	O(1)	Changed in place, priority kept.
Modify (new price)	O(log n)	Cancel plus a new submit.
Order book levels	O(n + L log L)	Scan the heap, then sort L price levels.
Audit add / read last k	O(1) / O(k)	Circular buffer.
Sorted-array insert (benchmark)	O(n)	Binary search is O(log n), but `splice` shifts elements.

Space: O(N) for N live orders across all books, plus the trade history. The audit trail is fixed at 2,000 entries, so it uses O(1) space.

9. User interface

The interface is a dark trading dashboard built with React. Green marks buys and price rises; red marks sells and falls. Users first see a landing page for a demo account. After entering, a live ticker tape runs across the top, and the header shows an MX60 index, market start/pause, a speed slider and reset, plus live counters for orders per second, trades, pending orders and fill rate.

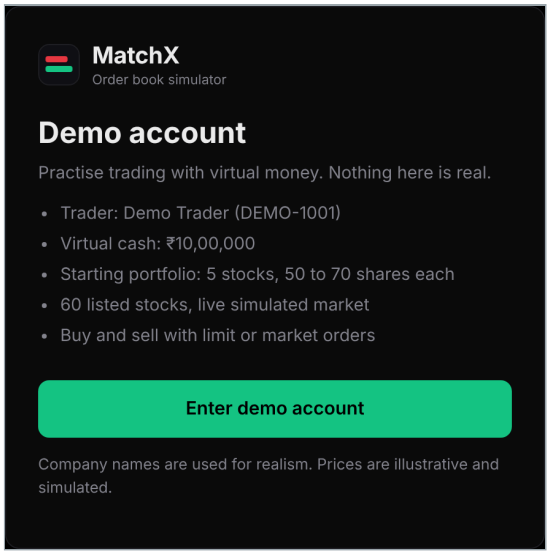


Figure 2 – Landing page: demo account with ₹10,00,000 virtual cash and a starting portfolio of 5 stocks.

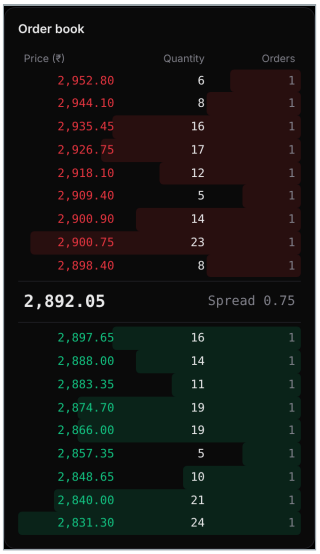


Figure 3 – Order book: best 9 asks (red) and bids (green), last price, spread, and depth bars.

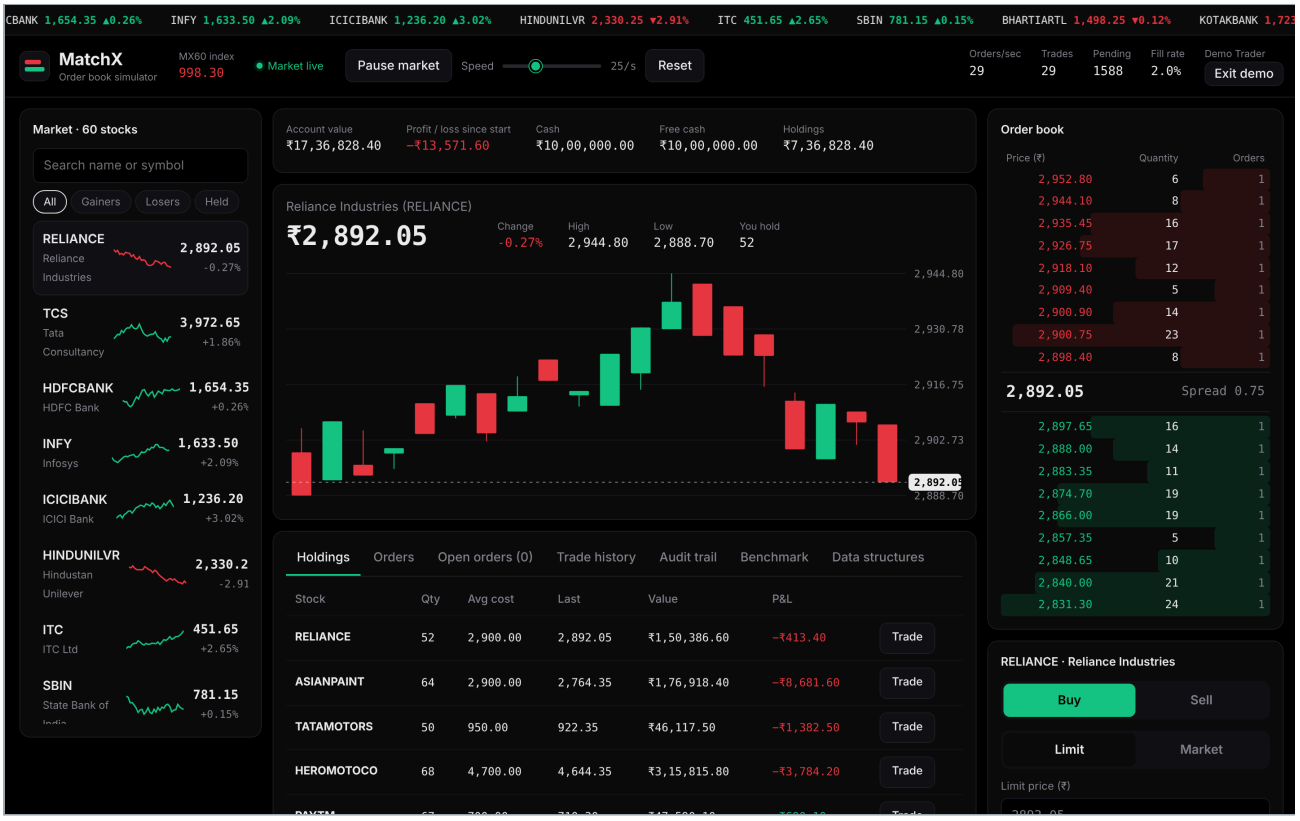


Figure 4 – Main dashboard: watchlist of 60 stocks with sparklines (left), account summary and candlestick chart (centre), order book and order ticket (right), and tabs below.

The **order ticket** switches between Buy/Sell and Limit/Market. It has quick 25–100% quantity buttons and shows the estimated value and the maximum the user can buy or sell. A **confirmation dialog** always appears before an order is sent. The tabs show holdings with profit/loss, all orders with status, open orders that can be updated or cancelled in place, trade history, the audit trail, the benchmark, and a table of data structures.

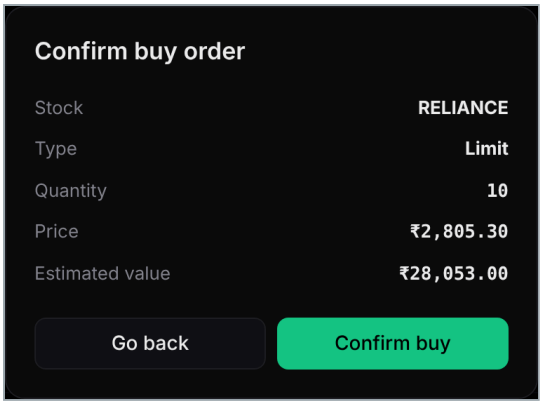


Figure 5 – Confirmation dialog for a limit buy of 10 RELIANCE.

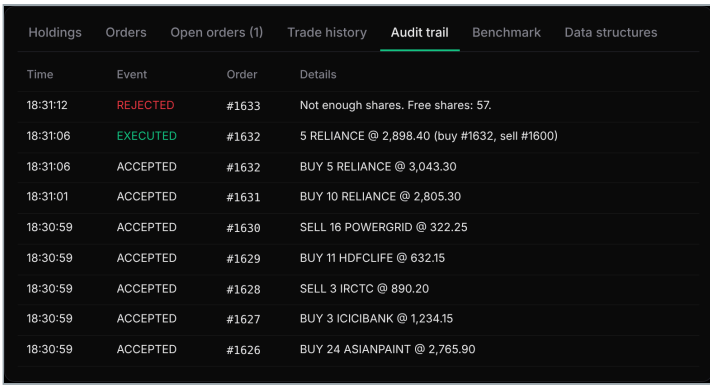


Figure 6 – Audit trail: a rejected sell, an executed market buy, and accepted orders.

10. Testing

10.1 Engine tests

The engine classes were taken unchanged from `MatchX.html` and run in Node.js (v22) with 17 scripted tests. **All 17 passed.**

#	Test	Observed result
1	Min-heap pops in sorted order	**PASS** – 1,2,3,5,7,8,9
2	Best (lowest) ask matches first	**PASS** – traded at 100
3	Same price: earlier order (FIFO) matches first	**PASS** – sell #1 (first was #1)
4	Trade happens at the resting order price	**PASS** – buy limit 110 traded at 100
5	Partial fill leaves remainder resting	**PASS** – status PARTIAL, remaining 6
6	Non-crossing order rests in the book	**PASS** – trades 0
7	Market (IOC) order cancels the unfilled rest	**PASS** – filled 3, cancelled 7
8	Cancelled order is skipped lazily at the heap top	**PASS** – stayed in heap until reached, then dropped; traded at 102
9	Buy rejected when cash is insufficient	**PASS** – Not enough cash. Free cash is ₹10,00,000.00.
10	Sell rejected when shares are insufficient	**PASS** – Not enough shares. Free shares: 0.
11	Fractional quantity rejected	**PASS** – Enter a price above 0 and a whole-number quantity.
12	Your fill updates cash and holdings	**PASS** – cash -4,400, +10 shares

13	Cash locked by open orders is not reused	**PASS** – Not enough cash. Free cash is ₹1,00,000.00.
14	Reducing quantity keeps time priority	**PASS** – modified order still filled first
15	Changing price re-queues with a new ID	**PASS** – new #3, bot order (earlier) filled first
16	Audit ring buffer keeps only the newest entries	**PASS** – 5,4,3
17	Heap and sorted-array engines produce the same trades	**PASS** – 4993 trades each

10.2 Interface tests (in the browser)

Action	Message shown by MatchX
Limit buy 10 RELIANCE about 3% below the last price	"Order #1631 is waiting in the book at ₹2,805.30. You'll get an alert when it fills."
Market buy 5 RELIANCE	"Order #1632 filled in full." It traded at ₹2,898.40, within the 5% cap of ₹3,043.30.
Limit sell 1,00,000 RELIANCE (more than held)	"Not enough shares. Free shares: 57." Logged as REJECTED.
Open orders tab	The resting order was listed with Update and Cancel buttons.

11. Performance benchmark

The Benchmark tab generates a reproducible random order stream (a seeded linear congruential generator) and runs it through the heap engine and the sorted-array engine. The table shows the median of five runs in Chromium 153.

Orders	Heap (ms)	Sorted array (ms)	Speed-up	Trades (both engines)
10,000	2.6	2.4	0.9×	2,444
50,000	21.1	31.4	1.5×	12,555
1,50,000	107.4	299.4	2.8×	37,872

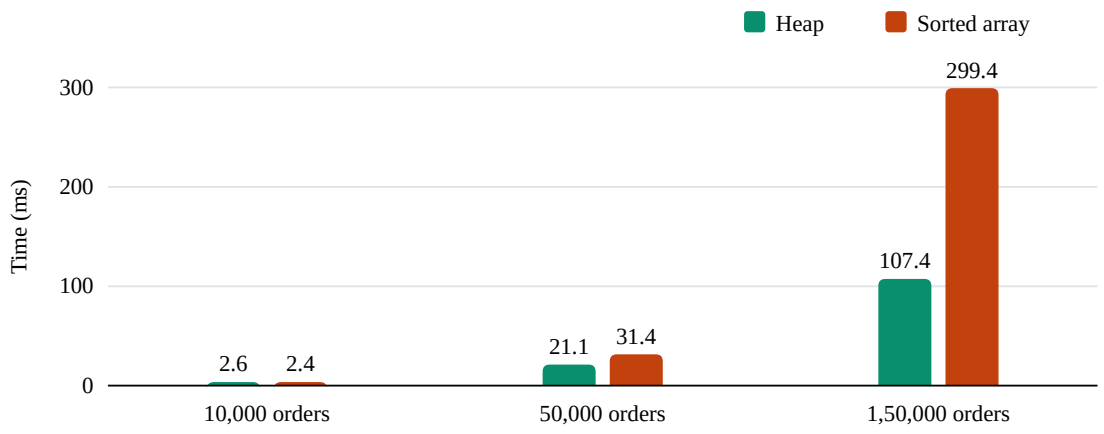


Figure 7 – Time to process the same order stream (median of five runs, Chromium). Lower is better.

At 10,000 orders the two engines take about the same time, because the books are small and `splice` is fast. As the books grow, every sorted-array insertion has to shift more elements (O(n)), while the heap stays at O(log n). At 1,50,000 orders the heap was **2.8× faster**. The gap was much larger in Node.js 22, where the sorted array took about 1.8–3.1 seconds against about 0.2 seconds for the heap. Exact timings depend on the device and the JavaScript engine, but in every run the heap engine scaled better. Both engines always produced identical trade counts, which shows they follow the same matching rules.

12. Limitations and future scope

12.1 Limitations

1. **No persistence.** Everything lives in memory and resets when the page reloads.
2. **Lazy-deleted orders occupy memory** until they reach the top of their heap, and `levels()` rescans the whole heap on every screen refresh.
3. **Bot orders are never cancelled**, so the number of pending orders keeps growing (about 1,600 after a few seconds of running). The trade list and price history also grow without limit.
4. **No self-trade prevention.** A user's buy can match the same user's resting sell. Real exchanges block this.
5. **Needs an internet connection** to load React, htm and the fonts from public CDNs.
6. Prices are simulated, and company names are used only for realism.

12.2 Future scope

- Add an indexed heap (position map), so a cancelled order can be removed immediately in O(log n).
- Add stop-loss and good-till-date orders, and self-trade prevention.
- Store the order book in a balanced tree or skip list keyed by price level, for fast depth queries.
- Save account state in local storage or a database, and add multi-user trading over WebSockets.

- Cap the trade history with a ring buffer, as is already done for the audit trail.

13. Conclusion

MatchX shows how a classic data structure, the binary heap, sits at the heart of a real-world system. Storing each side of every order book as a heap whose comparator encodes price–time priority keeps the best price available in constant time, and each match costs only logarithmic time. A hash map with lazy deletion makes cancellations cheap, and a circular buffer keeps the audit trail bounded. The 17 engine tests confirmed that the matching rules behave like a real exchange. The benchmark showed the heap scaling better than a sorted array as the order book grows. A full trading interface sits on top, which makes the data structures visible and easy to explore.